

จาก Prompt สู่ ระบบเกม

การพัฒนา Prompt สำหรับการโปรแกรมมิ่งโค้ดระบบเกม ด้วย Zero-shot
และ Few-shot

กรณีศึกษา: Coin Runner · HTML5 Canvas + JavaScript · 2026

โจทย์ของเรา คือทำให้ ไอเดียเกมเล่นได้จริง

2

รูปแบบ Prompt

5

องค์ประกอบ Game loop

7

กรณีทดสอบ

เราสร้างเกม 2D ที่ผู้เล่นเก็บเหรียญ หลบสิ่งกีดขวาง มีคะแนน พลังชีวิต
และสถานะชนะ-แพ้

เริ่มด้วยคำสั่งสั้น ๆ

ZERO-SHOT

ช่วยเขียนโค้ดเกม 2D ด้วย HTML, CSS และ JavaScript
ให้ผู้เล่นควบคุมตัวละครเพื่อเก็บเหรียญและหลบสิ่งกีดขวาง
มีคะแนน มีพลังชีวิต และแสดงข้อความเมื่อชนะหรือแพ้
ขอให้เปิดเล่นได้ในเว็บเบราว์เซอร์

ยังไม่มีตัวอย่างโค้ด ไม่มีโครงสร้างฟังก์ชัน และไม่มีกรณีทดสอบ



ต้นแบบเกิดขึ้นเร็ว แต่ยังต้อง **ตรวจสอบ**

ผลลัพธ์แรก

สิ่งที่ได้

- Canvas สำหรับพื้นที่เล่น
- ตัวละครและการรับปุ่ม
- ตัวแปรคะแนน/พลังชีวิต
- วงจร requestAnimationFrame

สิ่งที่ยังไม่ชัด

ระบบ Collision อาจไม่ครบ การจำกัดขอบเขตยังไม่แน่นอน และยังไม่มีการกำหนด Game state หรือวิธีทดสอบอย่างเป็นระบบ

บทเรียน: โค้ดจาก AI ต้องอ่าน ทดลอง และตรวจสอบก่อนใช้งาน

เพิ่มตัวอย่าง เพื่อ **คุมรูปแบบคำตอบ**

FEW-SHOT

คุณเป็นนักพัฒนาเกมเว็บ สร้างเกม Coin Runner
ตามรูปแบบ Input → Update → Collision → Render
มีเหรียญ 5 ชิ้น ชีวิต 3 หน่วย คะแนน และ Restart
แยกฟังก์ชัน setupGame, update, draw และ gameLoop
พร้อมกรณีทดสอบก่อนส่งคำตอบ



ผลลัพธ์ใหม่มี ระบบที่ตรวจสอบได้

01

Input

รับ WASD/Arrow

02

Update

ขยับและจำกัดขอบเขต

03

Collision

ตรวจเหรียญ/สิ่งกีดขวาง

04

State

คะแนน ชีวิต ชนะ แพ้

05

Render

วาด Canvas + HUD

ผลลัพธ์: เหรียญ 5 เหรียญ × 10 คะแนน, ชีวิต 3 หน่วย, มี Restart และมีกรณีทดสอบการทำงาน

ความสัมพันธ์ที่ทำให้ เกมขยับได้

COIN RUNNER MODEL

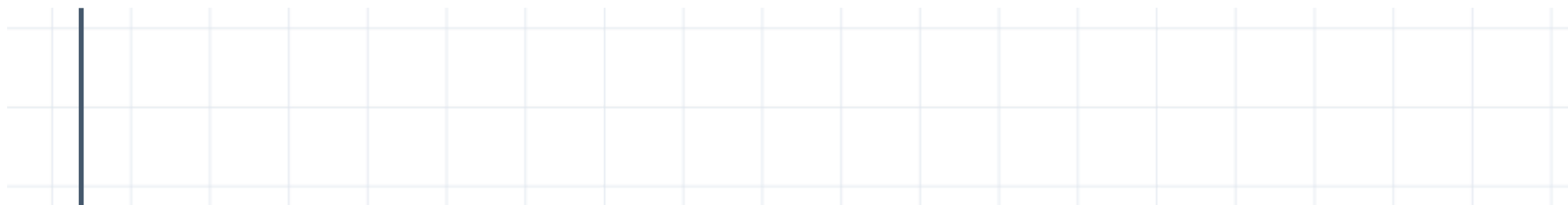
จากพิกัดและความเร็วสู่ Collision

ตำแหน่งใหม่เกิดจากตำแหน่งเดิม ความเร็ว และช่วงเวลาในแต่ละเฟรม

$$\begin{aligned}x_{t+1} &= x_t + v_x \Delta t \\ y_{t+1} &= y_t + v_y \Delta t\end{aligned}$$

Input: WASD / Arrow keys → **Update:** เปลี่ยน x, y → **Collision:** ตรวจสอบทับซ้อน

ผลลัพธ์: เก็บเหรียญเพิ่ม Score และชนสิ่งกีดขวางลด Lives



เปลี่ยน Code เป็น Diagram

flowchart LR

A[รับ Input] --> B[Update]

B --> C[ตรวจ Collision]

C --> D[อัปเดตคะแนน/ชีวิต]

D --> E[Render Canvas]

E --> F{ชนะหรือแพ้?}

F -- เล่นต่อ --> A

F -- จบเกม --> G[Restart]



จาก LaTeX เป็น บทความบนเว็บ

บทความ

คณิตศาสตร์เบื้องหลังการเคลื่อนที่ในเกม 2D

บทคัดย่อ. อธิบายความสัมพันธ์ของตำแหน่ง ความเร็ว และเวลาใน Game loop รวมถึงหลักการตรวจการชนด้วยกรอบสี่เหลี่ยม

สมการหลัก

$$x_{t+1} = x_t + v_x \Delta t$$

```
\documentclass{article}
\usepackage{amsmath}
\section{การเคลื่อนที่}
\begin{math}
x_{t+1}=x_t+v_x\Delta t
\end{math}
\section{การชน}
\begin{math}
x_1<x_2+w_2
\end{math}
\end{document}
```

จากผลลัพธ์แรก สู่ผลลัพธ์ใหม่

ประเด็น	Zero-shot	Few-shot
คำสั่ง	แนวคิดเกมโดยรวม	เทคโนโลยี โครงสร้าง และกติกา
โค้ด	AI เลือกโครงสร้างเอง	Input → Update → Collision → Render
การตรวจสอบ	ต้องค้นหาข้อผิดพลาดเอง	มีกรณีทดสอบชนะ แพ้ คะแนน และ Restart

Prompt ที่ดีไม่ใช่ Prompt ที่ยาวที่สุด แต่คือ Prompt ที่ระบุสิ่งที่ต้องการ วิธีการทำงาน และวิธีตรวจสอบได้ชัดเจน

AI ช่วยร่างโค้ด แต่คนยังต้อง ทดสอบระบบ

ร่าง

เริ่มจาก Zero-shot เพื่อสำรวจไอเดีย

พัฒนา

ใช้ Few-shot เพิ่มตัวอย่างและกติกา

ตรวจสอบ

ทดลองเล่น ตรวจสอบโค้ด และปรับปรุง